ComputeQuote: Open Cost Modeling for AI Training & Inference

CS146S Aut26 Final Project Proposal · Oct 1, 2026 · @Yogesh Seenichamy

ComputeQuote is a public web tool that predicts what an AI training or inference workload will cost. From a model architecture, token budget and GPU type, it finds a parallelism config that fits in memory, predicts step time with an analytical performance model, prices the run against a live GPU price index, and validates its predictions against published training runs.

Problem Statement

Compute is the largest cost of building AI, yet teams plan runs worth $10K–$10M with spreadsheets using training FLOPs ≈ 6 × parameters × tokens ÷ peak throughput. That shortcut ignores what actually sets cost:

- **Memory feasibility:** whether weights, gradients, optimizer states and activations fit per GPU depends on precision, ZeRO stage, recomputation and TP/PP/DP degrees.
- **Achieved efficiency:** communication (all-reduce, all-to-all), pipeline bubbles and interconnect limits keep real MFU well below peak.
- **Fragmented prices:** GPU rental prices vary widely across hyperscalers, neoclouds and marketplaces, change monthly, and use incompatible units.

VRAM calculators only check fit, papers give formulas but no tool, and pricing pages don't compare. The result: wrong fundraising budgets, mis-sized GPU commitments, and self-host vs. API decisions made on intuition.

Target Users

User	Goal	Needs
ML engineer / research lead (primary)	Plan a pretraining or fine-tuning run	Feasible parallelism config, GPU count, wall-clock time, explicit assumptions
Founder or finance/ops lead	Budget compute; pick providers and commitments	Defensible dollar figures, provider comparisons, sensitivity analysis
Analyst or AI community member	Sanity-check "model X cost $Y" claims	Fast, transparent estimates with a public accuracy record

Business & Use Case Context

ComputeQuote is used at the planning stage: scoping a run, writing a pitch deck's compute line, negotiating a GPU contract, or deciding self-host vs. API. The free calculator and price index drive distribution through shareable links on X; revenue could come from historical price-data API access, team workspaces, and procurement referrals.

Success criteria

- **Accuracy:** GPU-hours within ±25% of reported figures for ≥5 public runs (e.g., DeepSeek-V3, Llama 3).
- **Correctness:** cost math exact to the cent; feasibility verdicts consistent with published training configs.
- **Speed and freshness:** config search in under 1 s; prices from ≥8 providers refreshed daily, stale entries flagged.
- **Usability:** landing page to shareable estimate in under 2 minutes; links reproduce scenarios exactly.

Planned Features

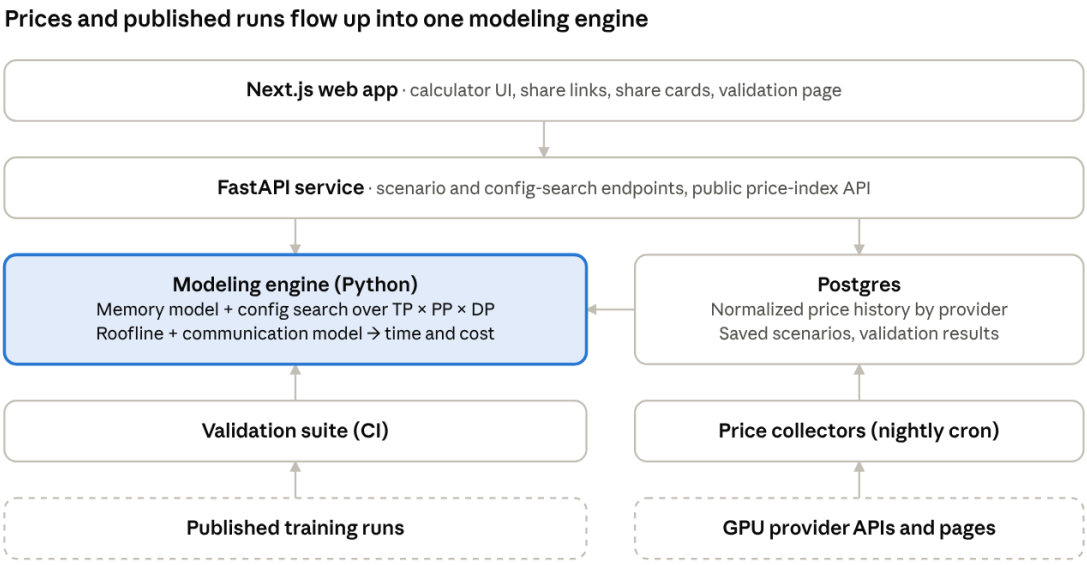
MVP (alpha, Nov. 5)

1. **Model spec input:** dense transformer architecture with presets for common open models.
2. **Memory model:** per-GPU weights, gradients, optimizer states and activations across BF16/FP8, ZeRO 0–3, recomputation, and TP/PP/DP sharding.
3. **Config search:** enumerate (TP, PP, DP, micro-batch), prune configs that exceed memory, rank by predicted throughput.
4. **Performance model:** FLOPs per step, roofline compute time, TP/DP all-reduce and PP bubble costs, compute/communication overlap.
5. **Cost and sharing:** GPU-hours, wall-clock and dollars from a curated price table; scenario encoded in a shareable URL.

Stretch (final, Dec. 10)

- **Inference mode:** prefill/decode throughput, KV-cache-limited batch size, $ per M tokens, break-even vs. API pricing.
- **MoE support:** active vs. total parameters, expert parallelism, all-to-all cost.
- **Live price index:** nightly collectors, normalization to $/GPU-hr and $/effective TFLOP, price history, staleness and outlier detection, public JSON API.
- **Polish:** validation page, auto-generated share cards, sensitivity analysis.

Architecture & Stack



ComputeQuote architecture · 6 components, 2 external sources

Dashed boxes are external data sources. The highlighted engine is a pure Python library with no I/O, so it is fast to test and easy for coding agents to work on in isolation. **Stack:** Python (Pydantic, NumPy) · FastAPI · Postgres · Next.js on Vercel · Fly.io · GitHub Actions · pytest + Hypothesis.

AI Tool Usage Plan

We keep one spec for humans and agents: `MODELING.md` (every equation, unit and invariant), `CLAUDE.md` (architecture and conventions), and a prompt log (`docs/prompts.md`) kept from day one.

- **Code generation:** Claude Code implements engine modules one equation group at a time from `MODELING.md`, then refactors the config search for speed.
- **Testing:** AI-generated property-based tests (Hypothesis) for invariants (e.g., per-GPU weight memory never rises with TP; feasible configs never exceed memory), plus human-checked golden tests on published runs; ≥85% engine coverage.
- **Code review:** AI PR review (Claude Code GitHub Action) with a unit-consistency checklist; humans review every equation change.
- **UI/UX:** AI-generated calculator and share-card prototypes, tested with ML engineers before porting to Next.js.
- **CI/CD:** AI-authored GitHub Actions for tests, preview deploys, nightly price collection, and a validation job that fails on accuracy regressions.

Communication & Final Presentation

Our 5–8 minute demo-day talk is built around one live scenario:

1. **Context (1 min):** why compute cost estimates go wrong, and for whom.

2. **Demo (3 min):** estimate a 70B training run end to end, open its share link, show the validation page.
3. **AI integration (1–2 min):** our spec-driven agent workflow, with before/after prompts from the log.
4. **Reflection (1–2 min):** results vs. our success criteria, where AI tools helped or failed, and the roadmap.